Proiectare Logică

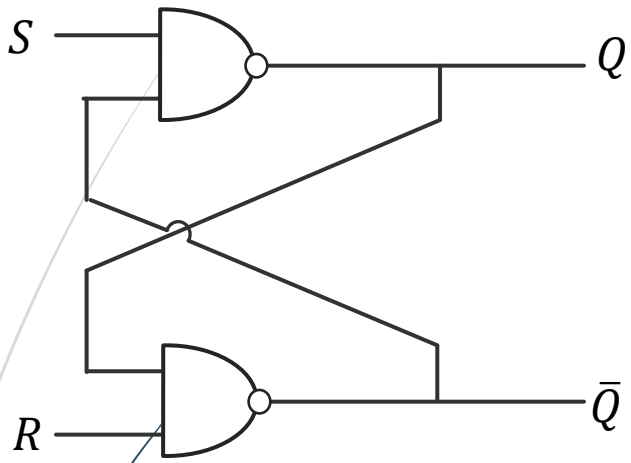
- I CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- **I CC: prof. dr. ing. Anca Morar**
  - anca.morar@cs.pub.ro

Curs 7: Circuite basculante bistabile.

Circuite basculante bistabile (CBB)

- Circuitele secvențiale care prezintă două stări stabile diferite se numesc circuite basculante bistabile (CBB). Ele sunt cele mai cunoscute elemente de memorare digitală.
- Circuitele basculante bistabile se diferențiază în principal în funcție de:
  - Numărul de intrări de comandă,
  - Frontul pe care au loc tranzițiile,
  - Modul de funcționare (sincron/asincron) şi
  - Funcția de stare realizată.

CBB fundamental – Latch RS



- Ce fel de circuit este?
- Secvențial
- De ce e secvențial? Avem feedback?
- Care e feedback-ul?
- Ieșirea depinde de momentul de timp
- Încercăm să îi facem tabela de adevăr
- Reprezentăm sub formă de diagramă Karnaugh

R	S	Q	!Q
0	0	1*	1*
0	1	0	1
1	0	1	0
1	1	Nu se modifică starea anterioară	

stare nepermisă

S \ R	0	1
0	-	1
1	0	Q^t

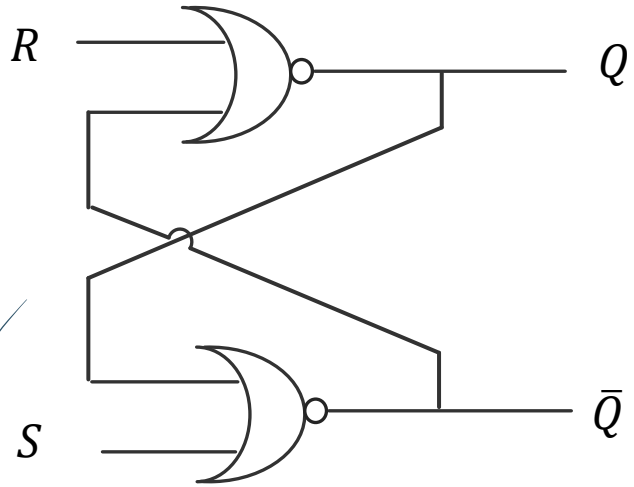
$$Q^{t+1} = RQ^t + R\bar{S}$$

CBB fundamental – Latch RS

- Observație: la circuitele combinaționale am învățat circuite cu ieșiri multiple, dar acolo ieșirile erau independente între ele. Aici, Q și $!Q$ sunt legate una de alta.
- Latch RS = circuit flip-flop = circuit secvențial fundamental asincron (ieșirea se modifică imediat ce se modifică intrările)
- Circuitul funcționează corect dacă nu există combinația $R = S = 0$ → Combinația aceasta este NEPERMISĂ → proiectantul circuitului trebuie să se asigure că cele 2 valori nu sunt simultan 0 pentru că altfel circuitul este neprotejat și are funcționare imprevizibilă (poate lua valori în mod aleator, poate să se blocheze într-o anumită stare etc.)
- Pentru $R = S = 1$, ieșirea depinde de starea anterioară

CBB fundamental – Latch RS

- Implementăm circuitul cu SAU-NU:



R	S	Q	$\bar{Q}$
0	0	Nu se modifică starea anterioară	
0	1	1	0
1	0	0	1
1	1	0*	0*

stare nepermisă

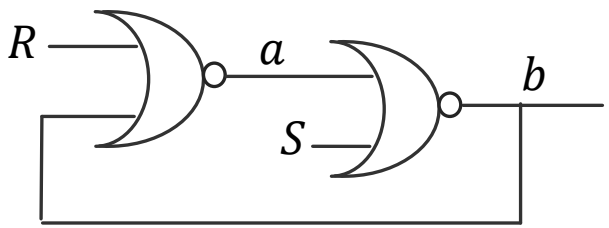
- Dualitatea dintre forma disjunctivă și cea conjunctivă

$$Q^{t+1} = !RS + !RQ^t$$

S \ R	0	1
0	Q^t	0
1	1	-

CBB fundamental – Latch RS

- De reținut: circuitele flip-flop sunt circuite asincrone (nu au intrare de ceas), sunt realizate folosind circuite combinaționale → sunt circuite secvențiale de ordin 1
- Care e valoarea lui Q ?
- Redesenăm circuitul:



$$a = \overline{R + b}$$

$$b = \overline{S + a}$$

$$a = \overline{R + \overline{S + a}}$$

$$a = \bar{R} \cdot (S + a) = \bar{R}S + \bar{R}a$$

$$b = \overline{S + \overline{R + b}} = \bar{S} \cdot (R + b) = \bar{S}R + \bar{S}b$$

CBB fundamental – Latch RS

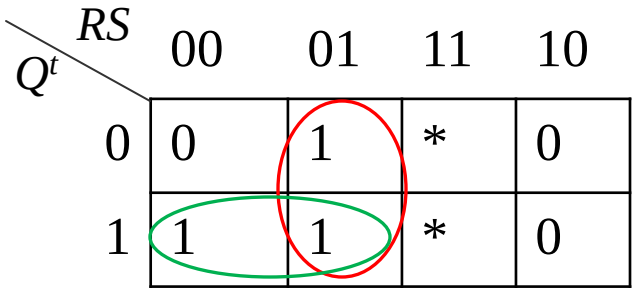
- Observație: sunt 2 momente de timp (t și $t+1$):
- Pe baza circuitului: $Q^{t+1} = !RS + !RQ^t$
- Cum Q^{t+1} depinde și de Q^t, facem tabelul de adevăr care să includă și Q^t

R	S	Q	$\overline{Q}$
0	0	Nu se modifică starea anterioară	
0	1	1	0
1	0	0	1
1	1	0*	0*

stare nepermisă

R	S	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	Nepermisă
1	1	1	Nepermisă

Latch RS – ecuația de stare următoare



Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

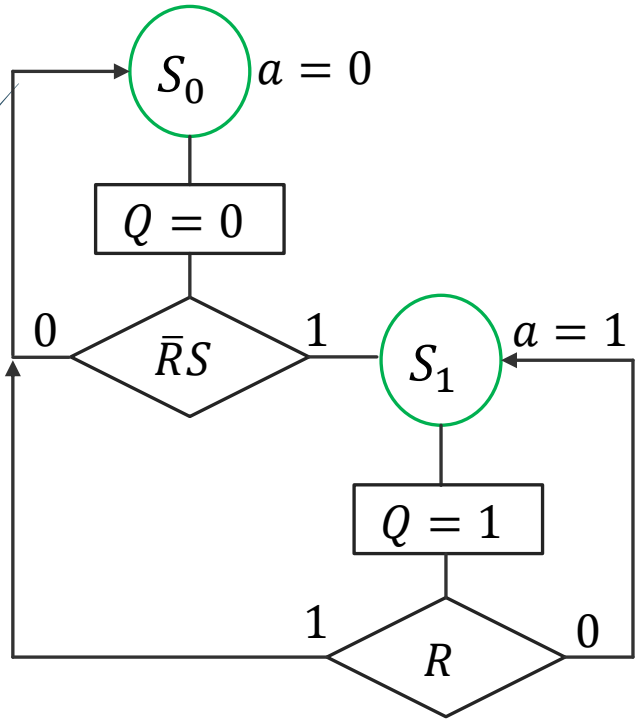
- $Q^{t+1} = !RS + !RQ^t$
- Ecuația de stare următoare → descrie funcționarea circuitului
- Câte stări are circuitul?
- 2
- Câte variabile de stare avem?
- 1 → Q este variabila de stare ce ia valoarea 0 sau 1

Organigrama circuitului - Latch RS

- Cum o construim? Fie pornind de la ecuația de stare, fie de la tabelul de adevăr, fie de la diagrama Karnaugh

$$Q^{t+1} = !RS + !RQ^t$$

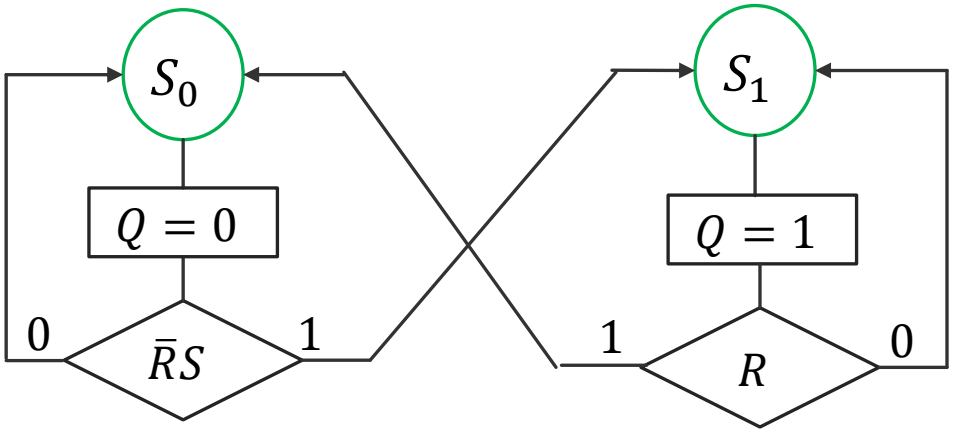
Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0



R	S	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	Nepermisă
1	1	1	Nepermisă

Organigrama circuitului - Latch RS

- Redesenăm:



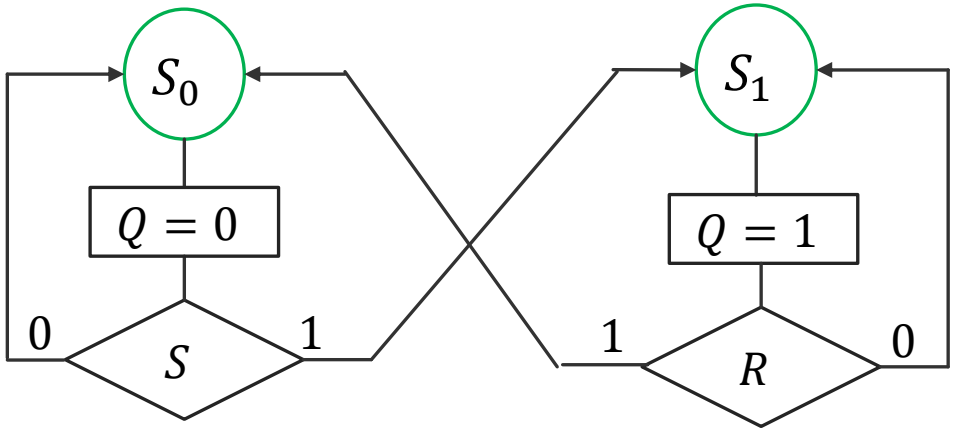
- Observație: Organigrama are o simetrie geometrică, dar prezintă asimetrie dpdv al controlului tranziției: S_0 e controlată de ! RS în timp ce S_1 e controlată de R

Organigrama circuitului - Latch RS

- Dacă se interzice simultaneitatea $R = S = 1$ astfel încât Q și $!Q$ să fie consistente.

Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

- $Q^{t+1} = S + !RQ^t$



Diagrame Karnaugh - Latch RS

- Am făcut diagrama Karnaugh în care R,S au fost variabile

S \ R	0	1
0	Q^t	1
1	0	-

Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

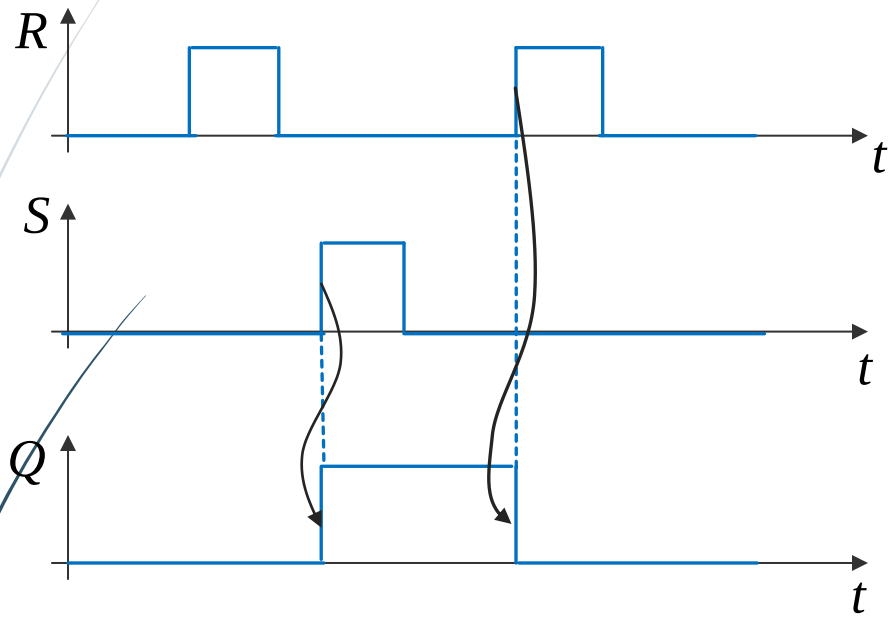
Mealy pentru că Q^{t+1} depinde și de stări și de intrări

- Apoi am adăugat Q
- Putem face și invers? În care să avem doar dependența Q^{t+1} de Q^t și RS să fie interioare
- $Q^{t+1} = S + !RQ^t = SQ^t + S!Q^t + !RQ^t$
- Care e de tip Mealy și care e de tip Moore și de ce?

Q^t	0	1
	S	$S + !R$

Moore pentru că ieșirea Q^{t+1} depinde doar de stări

Diagrama de semnal - Latch RS



- R – reset
- S – set
- Flip-flop elementar în logica pozitivă
- R aduce circuitul în 0, iar S în 1
- Variabila de stare este și variabilă de ieșire

Ordinele automatelor

- Un automat de ordin $n+1$ poate fi generat prin interconectarea unor automate de ordin inferior din care cel puțin unul este de ordin n și e conectat într-o buclă de reacție
- Automatul de ordin 0 este un circuit combinațional și e caracterizat prin absența variabilelor de stare
- Prin conectarea în serie sau în paralel a mai multor automate de ordin n se obține tot un automat de ordin n
- Până acum: orice circuite combinaționale cuplate în serie sau în paralel, dar fără reacție, formează tot un circuit combinațional

Ordinele automatelor

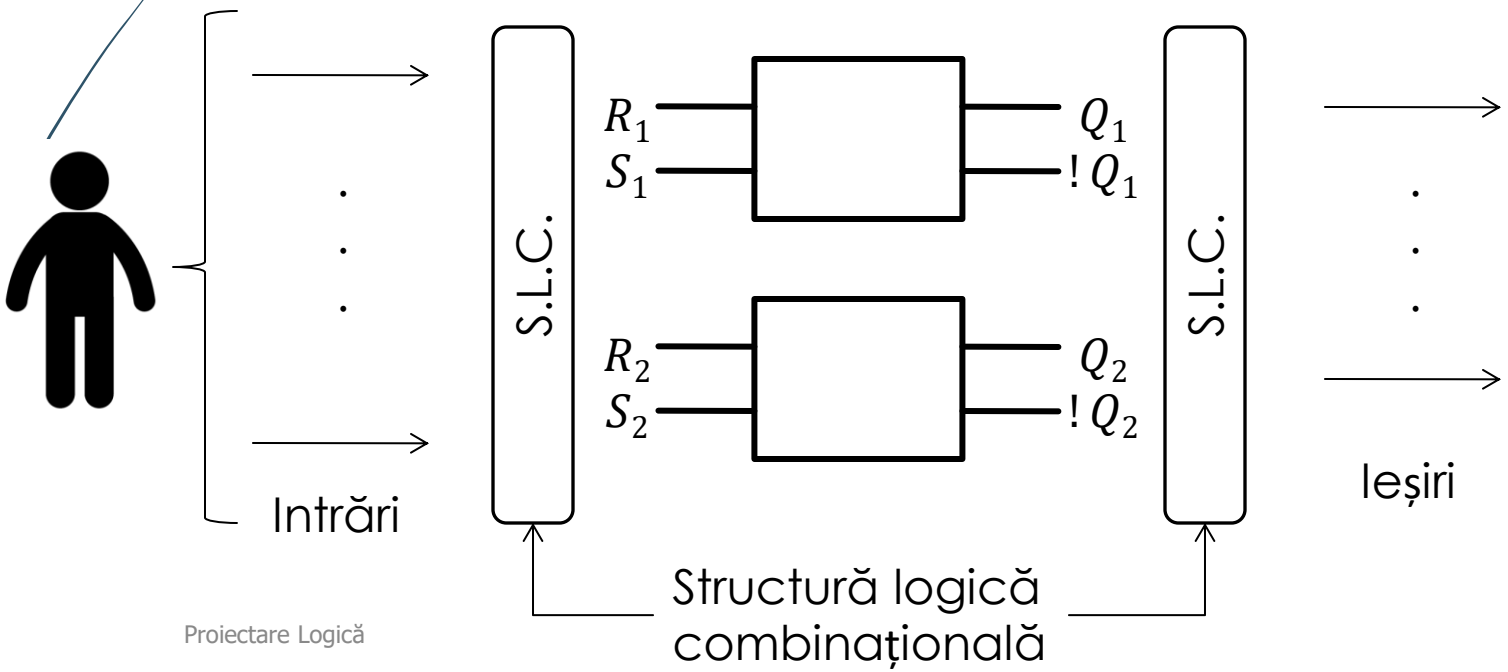
- Flip-flop am spus că este de ordin 1 – de ce?
- Pentru că am buclă de reacție
- Dacă flip-flop îl punem în bucla de reacție a altui circuit → circuit de ordin 2
- La curs vom lucra în special cu circuite de ordin 2, dar spre sfârșitul cursului o sa facem circuite programabile care sunt de ordin 3 sau mai mare.
- Circuitele programabile sunt caracterizate prin elemente de soft care sunt scrise în circuit → reacția se face prin soft (reacția se face prin acces la o anumită zonă de memorie – se revine într-un anumit punct al procesării).

Reprezentarea circuitelor Flip-Flop

- Reprezentare



- Dacă am mai multe, le dau indici pentru a le identifica



Utilizarea circuitelor Flip-Flop

Q_1	Q_2	$Stare$
0	0	S_1
0	1	S_2
1	0	S_3
1	1	S_4

- Câte stări are circuitul?
- 4
- Ce fac cu ele? La ce sunt bune?
- Comand diferite dispozitive (ex: închid/deschid uși, aprind/închid beculețe, ud gazonul, închid/deschid ecluze etc.)
- S.L.C. – urile, pe baza unei anumite logici, iau intrările și le aplică la intrările circuitelor secvențiale astfel încât la ieșire se generează stările în secvența și la momentul potrivit

Circuite secvențiale

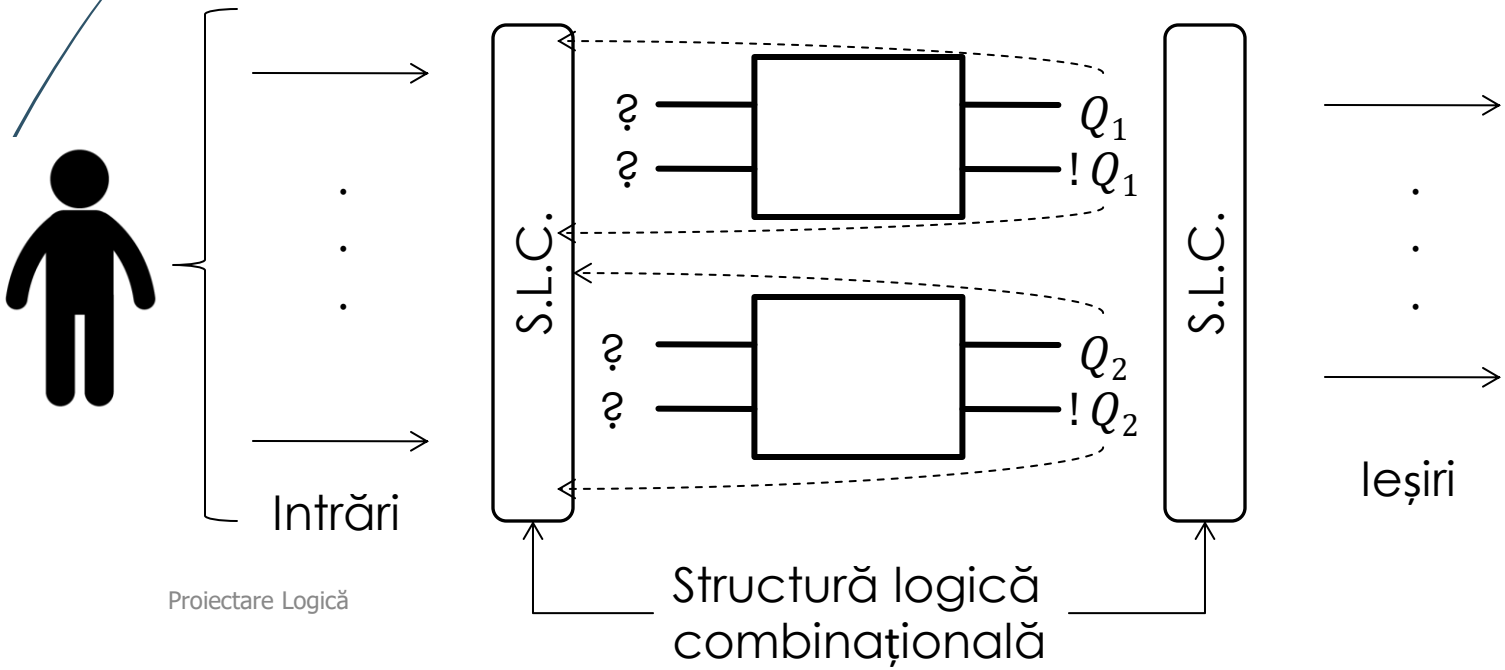
- Ce știm acum?
  - Ce funcții combinaționale să aplicăm ca să obținem rezultatele dorite → facem la fel și la circuitele secvențiale (intrările sunt stările + combinația variabilelor de intrare)
- Ce nu știm?
  - Cum să calculăm ce trebuie pus la intrarea circuitelor secvențiale astfel încât ieșirile să fie corecte
  - Nu e suficient să știm ce se întâmplă cu un singur circuit secvențial (CBB), ci trebuie să știm și ce interacțiuni există între ele

Circuite secvențiale

- Deci nu e suficient să definim
- $Q_1^{t+1} = f(Q_1^t, intrări)$
- $Q_2^{t+1} = f(Q_2^t, intrări)$
- Pentru că sunt scoase din context → mai intervine de fapt și cealaltă variabilă:
- $Q_1^{t+1} = f(Q_1^t, intrări, Q_2^t)$
- $Q_2^{t+1} = f(Q_2^t, intrări, Q_1^t)$
- În realitate, ieșirile de la Q vor fi trecute pe intrare tocmai ca să ne asigurăm că ținem cont de dependențele dintre variabilele de stare

Circuite secvențiale

- Crește ordinul circuitului secvențial → cum justificăm asta? → funcționalitatea depinde de un anumit algoritm (succesiune de acțiuni ce trebuie întreprinse pentru a ajunge la un anumit rezultat)
- Ex: un semafor la o intersecție simplă: verde → galben după un timp → roșu după un timp. La intersecția de la Leu e mai complex, avem un alt algoritm de urmat.



Circuite secvențiale

- Ex: dacă avem un circuit secvențial cu 7 stări → de câte circuite flip-flop am avea nevoie ca să îl implementăm?
- 3, față de exemplu anterior, am mai avea încă un RS și toate ar fi legate între ele prin faptul că intrările sunt realizate prin circuite combinaționale ce primesc intrările circuitului + ieșirile de la toate celelalte RS-uri
- Ce învățăm de fapt?
- Cum să combinăm intrările ca să obținem la ieșire ceea ce dorim (funcționalitatea dorită)

Sincronizarea circuitelor

- La circuite combinaționale nu are sens să vorbim despre sincronizare.
- La circuitele secvențiale, în funcție de modul de realizare a tranziției avem circuite sincrone/asincrone
- Ex: RS-ul făcut este asincron.
- Sincronizarea se face pe baza unei intrări de ceas (Tact, Clock)
- Există 2 tipuri de sincronizări:
  - Sincronizarea pe front
  - Sincronizarea de tip master slave

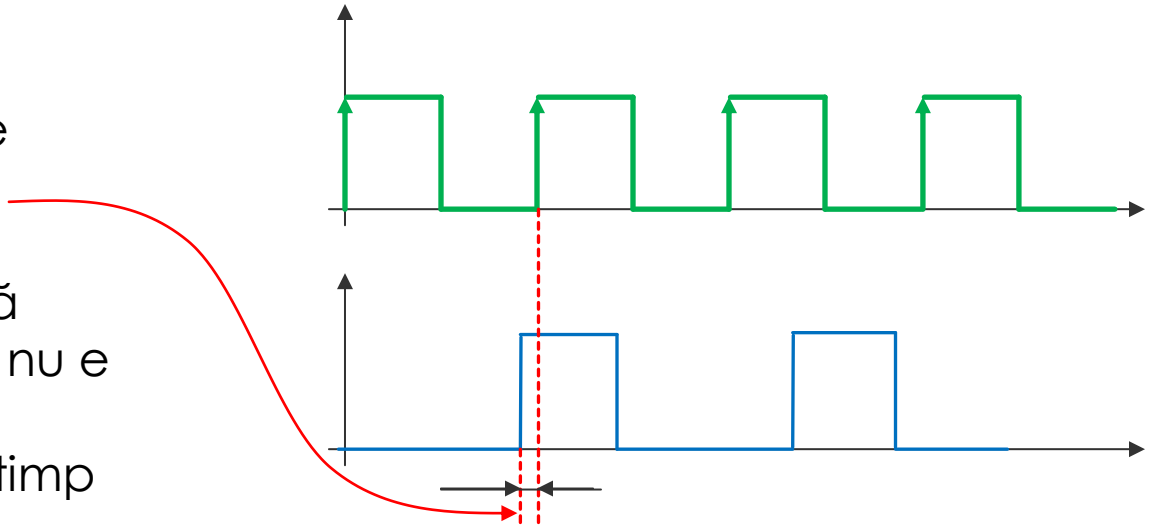
Sincronizarea pe front crescător / descrescător

- Pe unul din cele 2 fronturi are loc tranziția → se întâlnește la o serie de CBB-uri
- Realizează operația de basculare la o anumită valoare a tensiunii, atunci când are loc tranziția pozitivă/negativă
- Un dispozitiv dat nu recunoaște decât un singur tip de tranziție (fie pozitivă, fie negativă)
- Intrarea de ceas acționată pe front validează intrările de date și transferă datele sincron la ieșire

Sincronizarea pe front crescător / descrescător

- Avantaje:
  - Dacă se cunoaște momentul de timp se pot pregăti condițiile de intrare astfel încât să se realizeze tranziția când trebuie
  - În cataloage, circuitele au niște valori pentru timpul minim de stabilitate pentru a recunoaște o anumită schimbare a valorilor de intrare

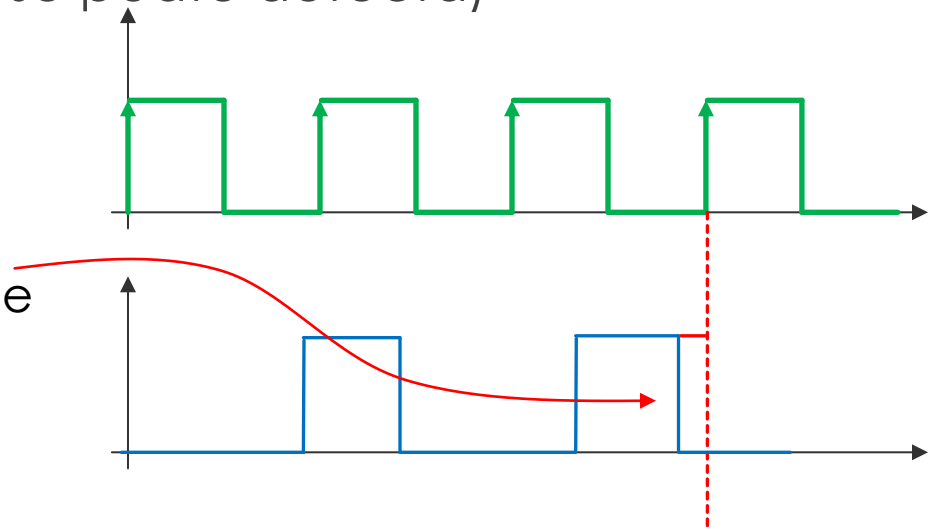
Aici nu se poate detecta pentru că semnalul nu e stabil de suficient timp



Sincronizarea pe front crescător / descrescător

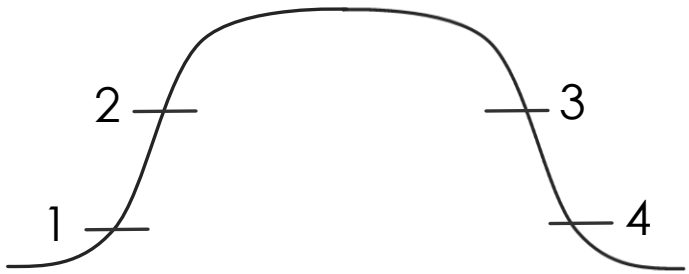
- Dezavantaje:
  - Nu mereu putem sincroniza intrările cu semnalul de sincronizare
  - Apare când mărim prea mult perioada semnalului de sincronizare
  - Există metode de rezolvare a problemelor (ex: folosire de circuite de memorare pentru prelungirea semnalului până se poate detecta)

Aici nu se poate detecta pentru că semnalul nu există pe frontul crescător al ceasului



Sincronizarea master-slave

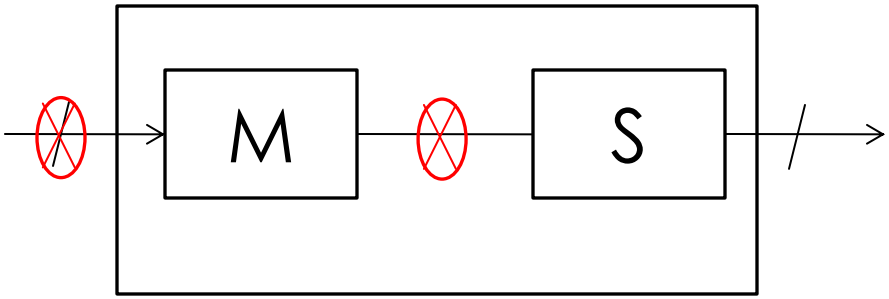
- Se numește și sincronizare în 4 timpi – avem 4 momente de timp în care se întâmplă ceva în circuit (semnalul de ceas acționează circuitul în 4 pași)
- E utilizată la acele circuite secvențiale la care putem identifica 2 module: master (M) & slave (S)
- Este format din 2 latch-uri în serie

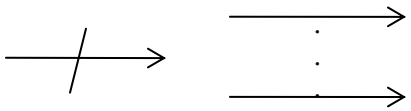


1. Izolează etajul S de M (ieșirile vechi rămân stabile)
2. Se validează intrările datelor în M
3. Se invalidează intrările de date (modificările noi nu se mai iau în calcul)
4. Se face trecerea datelor din M în S

Sincronizarea master-slave

- Observație: intrările nu sunt conectate niciodată direct la ieșiri, pentru orice moment al tactului.



-  1,2, ... N semnale
- Sincronizarea în 4 timpi protejează ieșirea de intrare (ex. La JK-uri)